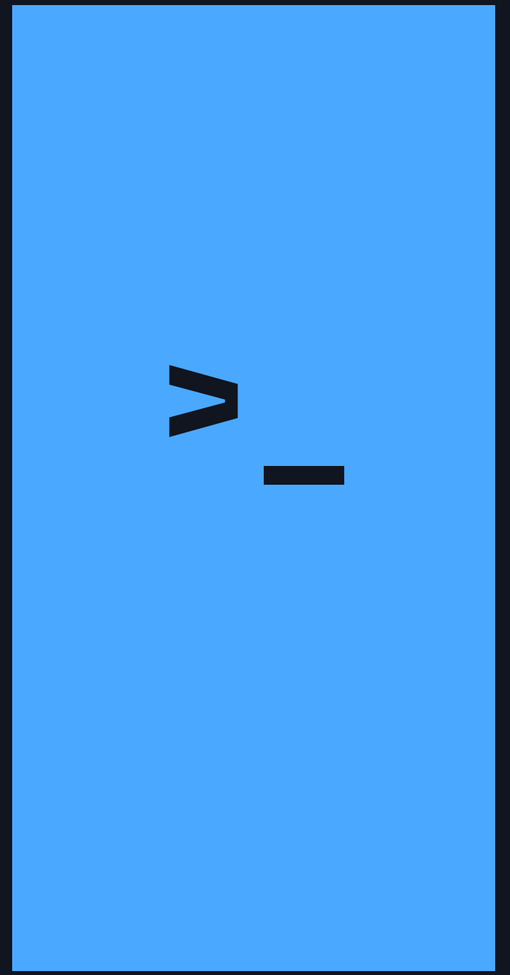
Python Basics 101



Writing clean, modern code.

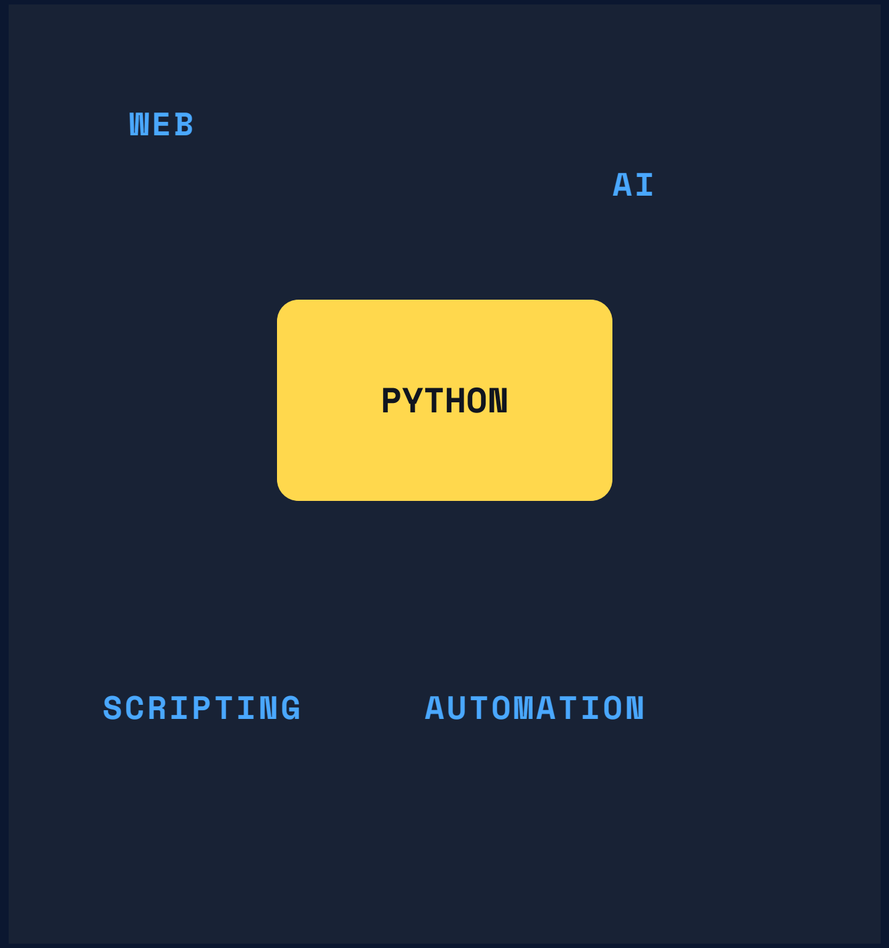
Nadav Sudri | OOP, AU

What is Python?

A high-level, interpreted, dynamically typed language.

Readable by design. Powerful enough for web development, AI, scripting, and automation.

EASE OF USE × READABILITY



Python vs. C / Java

C / JAVA

```
int x = 5;
```

compile → declare → execute

PYTHON

```
x = 5
```

write → run

Interpreted. Dynamic typing. Less ceremony.

The Zen of Python

“Beautiful is better than ugly.”

“Explicit is better than implicit.”

Write code that reads like plain English.

The Zen of Python

```
import this
```

The Zen of Python, by Tim Peters

Beautiful is better than ugly.

Explicit is better than implicit.

Simple is better than complex.

Complex is better than complicated.

Flat is better than nested.

Sparse is better than dense.

Readability counts.

Special cases aren't special enough to break the rules.

Although practicality beats purity.

Errors should never pass silently.

Unless explicitly silenced.

In the face of ambiguity, refuse the temptation to guess.

There should be one-- and preferably only one --obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Now is better than never.

Although never is often better than *right* now.

If the implementation is hard to explain, it's a bad idea.

If the implementation is easy to explain, it may be a good idea.

Namespaces are one honking great idea -- let's do more of those!

Slide 6

Indentation IS the Syntax

Whitespace defines blocks — no braces, no semicolons.

JAVA

```
if (ready) {
  run();
}
```

PYTHON

```
if ready:
    run()
```

Those four spaces are structural.

Pythonic Idioms

UN-PYTHONIC

```
for i in range(len(arr)):
    use(arr[i])
```

PYTHONIC

```
for item in arr:
    use(item)
```

UN-PYTHONIC

```
for i in range(len(arr)):
    if arr[i] == item:
       // do something
```

PYTHONIC

```
if item in arr:
   // do something
```

Name the thing you mean. Remove bookkeeping. Improve readability.

Slide 8

Variables & Types

Names point to objects. Types are discovered at runtime.

int

```
x = 10
```

whole numbers

float

```
price = 9.99
```

decimals

str

```
name = "Alice"
```

text

bool

```
is_active = True
```

logic

Strings & f-Strings

CODE

```
name = "Alice"
print(f"Hello {name}!")
```

OUTPUT

```
Hello Alice!
```

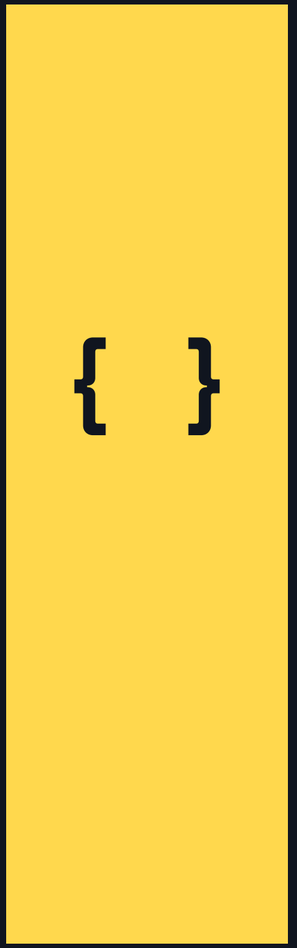
Prefix with f, then place expressions inside { braces }.

TASK 01

Introduce yourself with variables.

Store a name and age. Print a greeting with an f-string; add a condition that says whether the person is an adult. Keep the output on two lines.

hint → Compare age with 18, then format both values



Slide 11

Conditionals control flow

Python reads like a decision: if this, elif that, else otherwise.



and or not

A decision in code

PASS / FAIL

```
grade = 75

if grade >= 60:
    print("Passed")
else:
    print("Failed")
```

OUTPUT

Passed

Add elif when there is a meaningful middle case.

TASK 02

Even or odd?

Read an integer. Print EVEN or ODD; for 0 print ZERO. For a negative value, print INVALID instead. Use one if / elif / else chain.






```
hint  →  Check the special
cases before using % 2
```

Loops repeat with intent

for

Iterate directly over a sequence.

while

Repeat while a condition remains true.

`break` exits `continue` skips

Loop, skip, stop

CODE

```
for x in range(5):
    if x == 2:
        continue
    if x == 4:
        break
    print(x)
```

OUTPUT

```
0
1
3
```

continue skips 2. break stops before 4.

TASK 03

Print a number triangle

7

SKIP

Use nested for loops to print:
1
12
123
1234
12345
Ask for rows; row number sets the count.
Extra: count down on every second row.

```
Hint → The outer loop controls rows;
the inner loop controls each row.
```

Lists

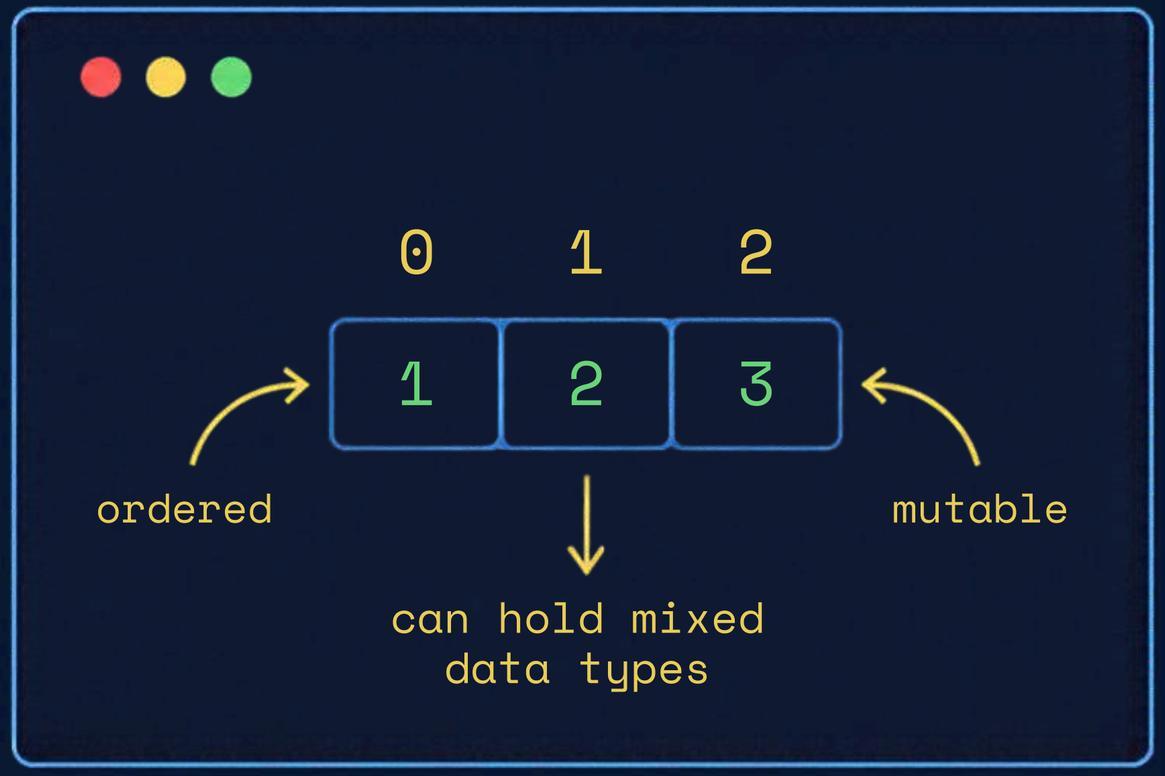
Lists are **ordered**, **mutable** collections that can hold **mixed data types**.

> Example 1:

```
[1, 2, 3]
```

> Example 2:

```
["Alice", 42, True]
```



SLIDE 18

List Operations

Common **operations** you'll use every day.

1 .append() — adds an item to the end

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)
```

→ `[1, 2, 3, 4]`

4 is added to the end

2 .pop() — removes and returns the last item

```
my_list = [1, 2, 3, 4]
removed = my_list.pop()
print(removed)
print(my_list)
```

→ `4` last item returned

→ `[1, 2, 3]` list after removal

3 Slicing — get a range with my_list[1:4]

```
my_list = [10, 20, 30, 40, 50]
slicePart = my_list[1:4]
print(slicePart)
```

→ `[20, 30, 40]`

items at index 1 up to 3



SLIDE 19

Pythonic List Operations

Combine lists, subtract values, and get unique items—the Pythonic way.



TAKEAWAY

Pythonic: use **+** to concatenate; use a list comprehension when order matters; use set(. . .) for unique values and set difference.



Python lists do not support the – operator directly.

```
numbers_a = [1, 2, 3]
numbers_b = [3, 4, 5]

# Add / concatenate
combined = numbers_a + numbers_b
# [1, 2, 3, 3, 4, 5]

# Subtract while preserving order
remaining = [x for x in numbers_a if x not in numbers_b]
# [1, 2]

# Set conversion and set difference
unique = set(numbers_a)
difference = set(numbers_a) - set(numbers_b)
# {1, 2}
```

SLIDE 20

TASK 04

Create 4 movies, append a fifth, then print the 2nd item and the last 2 as a slice. Use indexes—no movie names in print().



```python
# 1) Create a list of 3 favorite movies
movies = ["Inception", "Interstellar", "The Dark Knight"]

# 2) Append one more movie
movies.append("The Matrix")

# 3) Print the 2nd movie using an index
print(movies[1])
```

HINT

Check the slice length.

Tuples

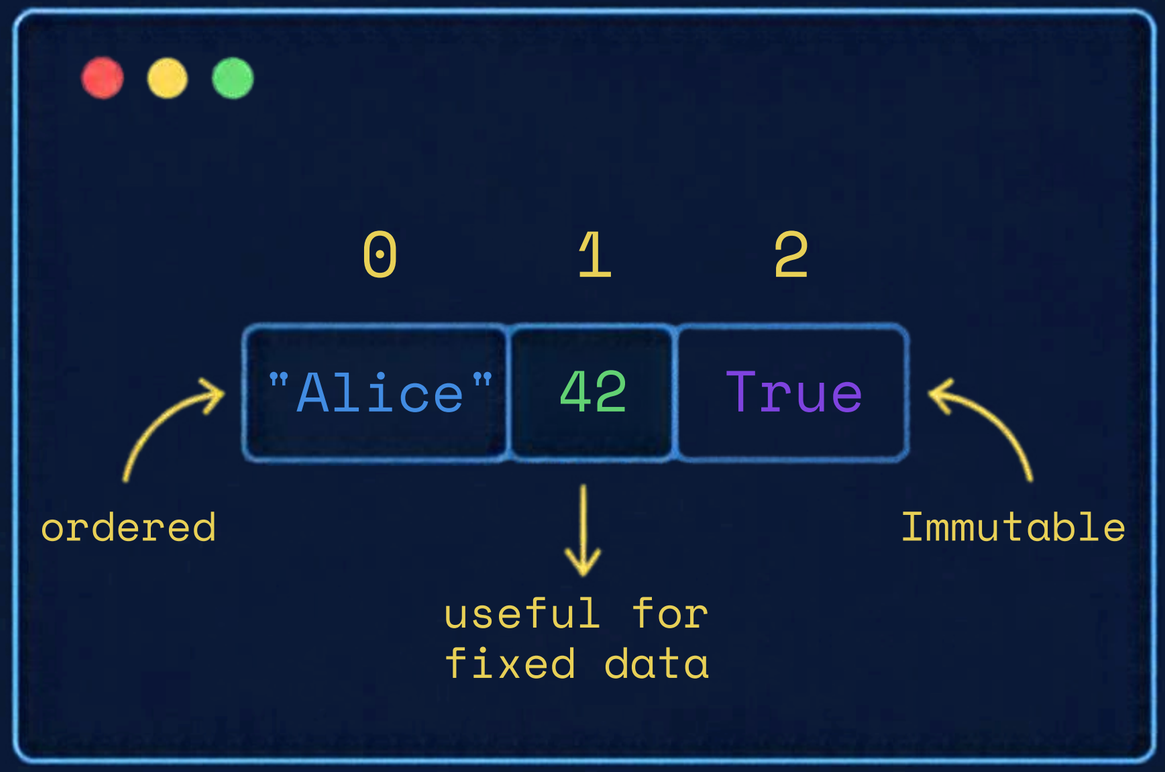
Tuples are **ordered** but **immutable**.

They are often **faster** than lists.

Tuples are **useful for fixed data**.

> Example:

```
("Alice", 42, True)
```



Tuples in Action



```
>>> coordinaates = (32.1, 34.8)
```

```
(32.1, 34.8)
```

```
>>> coordinaates[0]
```

```
32.1
```

```
>>> coordinaates[0] = 10
```

```
TypeError:
'tuple' object does not support item assignment
```

> Tuples are **immutable**:
> their values cannot be changed after creation.

TASK 05

Return name, age, and active as a tuple. Unpack it, uppercase the name, and print whether age is at least 18.

```
def get_info()
    name = "Alice"
    age = 25
    active = True
    return (name, age, active)

name, age, isActive = get_info()
```

💡 **HINT**

```
return (a, b, c)

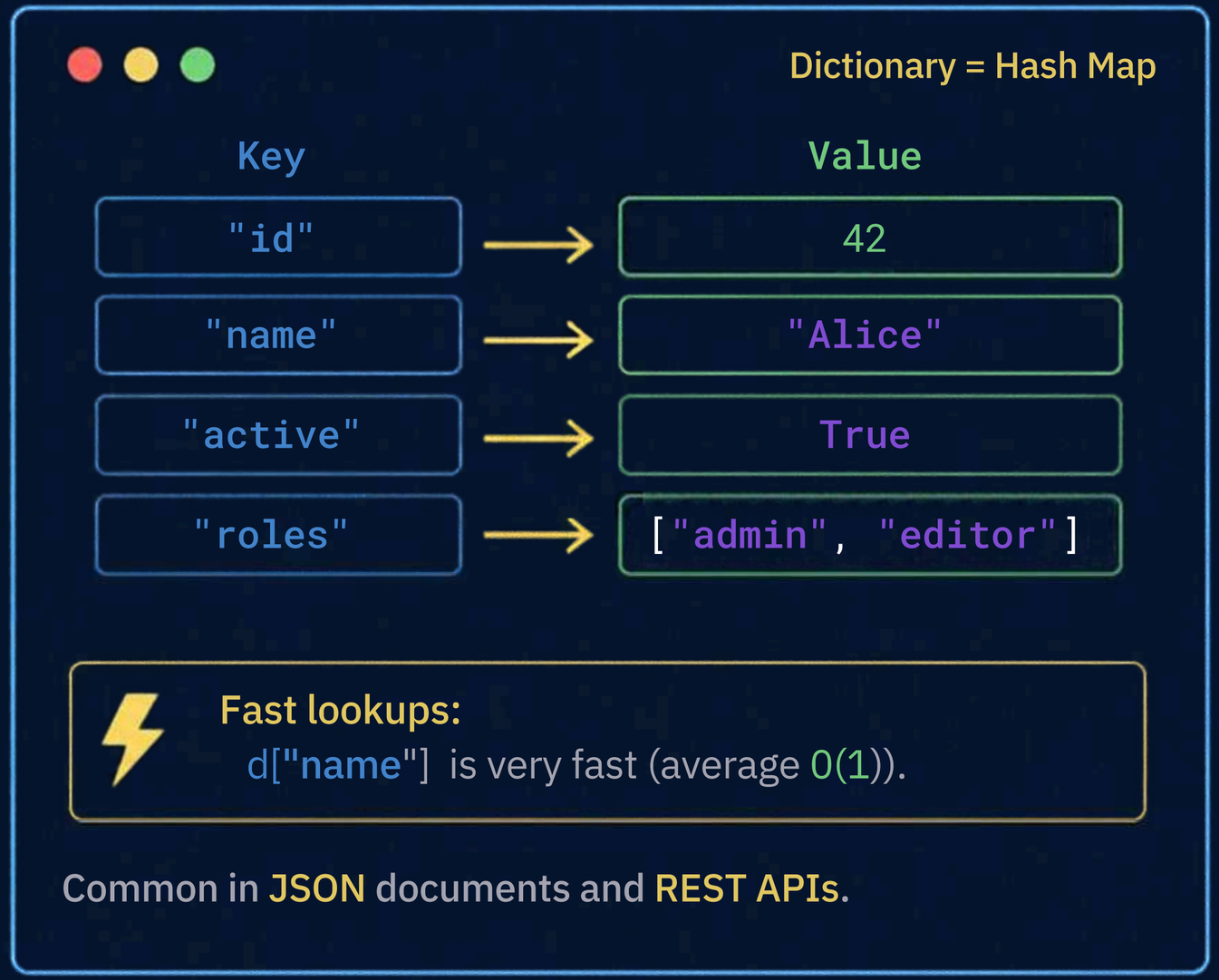
x, y, z = func()
```

Dictionaries

Dictionaries store **key-value pairs**.
Think of them as **hash maps**:
keys are **unique** and map to **values**.

> **Fast lookups** by key on average O(1)
> Ideal for **structured, labeled data**
> Widely used in **JSON** and **APIs**

```
> Example:
  user = {
 "id": 42,
 "name": "Alice",
 "active": True,
 "roles": ["admin", "editor"]
}
```



Dictionary Operations

- **Retrieve a value**
  Access a value using its **key**.

- **Add a new key-value pair**
  Assign a value to a **new key** to add it to the dictionary.

```
# Create a dictionary
student = {"name' "Ron', "grade': 95}
```

```
# Retrieve the value for the key "name"
student["name']
```

```
Output:  'Ron'
```

```
# Add a new key-value pair
student["passed"] = True

# Resulting dictionary
student
```

```
Output:  {'name' 'Ron', 'grade' 95, 'passed' True}
```

TASK 06



Create a car with nested owner data. If year < 2020, set it to 2024. Add color. Print owner and year.



TIP: Update the nested key, not the whole owner.

```
# 1. Create the dictionary
car = {
    "brand": "Toyota",
    "model": "Corolla",
    "year": 2019
}

# 2. Update the year to 2024
car['year'] = 2024

# 3. Print only the required
fields
```

```
Output: owner name + updated year
```

Sets

Sets contain unordered, unique items.

> Unordered: items have no guaranteed position.

> Unique: duplicates are automatically removed.

```
# Example: duplicates are removed
set([1, 2, 2, 2, 3])
```



```
{1, 2, 3}
```



Duplicates are removed; only unique values remain.

Functions

Functions let you **reuse code**.



Reusability
Write once, use many times.
Keeps your code **DRY**
(Don’t Repeat Yourself).



The **def keyword**
starts a function definition.
Give it a name, optional
parameters, and an **indented block of code**.



Better structure
Functions make programs easier
to read, test, and maintain.

example.py

```
1  def greet(name):
2      """Return a friendly greeting."""
3      return f"Hello, {name}!"
4
5  message = greet("Alice")
6  print(message)
7  # Output: Hello, Alice!
```



Define with **def**, run with a **call**.
Parameters go in, values come out (via **return**).
Small functions, **big impact**.

Parameters & Returns

Functions can take parameters
(with type hints and defaults)
and return values.

> Type hints improve readability

> Defalt values make arguments optional

> Return values give results back

```python
# Define a function with type hints,
# a default parameter, and a return value

def add(a: int, b: int = 5) -> int:
    return a + b

# Call the function
result1 = add(3)
result2 = add(10, 7)

# Show the results
print(result1)
print(result2)
```

OUTPUT

```
8
17
```

TASK 07

PROMPT:

Write calculate_area(width, height) with height defaulting to 4. Return 0 for non-positive dimensions. Test (6, 4) and one invalid call.

EXAMPLE OUTPUT:

```
Area: 24
Invalid
area: 0
```

```python
def calculate_area(width, height):
    """Return the area of a rectangle
    given width and height."""

    # Calculate area
    area = width * height

    # Return area
    return area


# Call the function and print the result
result = calculate_area(6, 4)
print(f"Area: {result}")
```



WHY HANDLE ERRORS?

Programs should not crash unexpectedly.

Errors can happen (bad input, missing files, dividing by zero, etc.). Without handling, your program stops and shows a traceback.

Exceptions can be caught and handled.

Use **try** and **except** to catch errors and decide what to do next.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Please enter a valid number.")
else:
    print("Result: {result}")
```

Simple idea:

Try something risky.
If it fails, handle the problem gracefully and keep the program running.

try / except

PROMPT:

Use try / except to catch a ValueError when converting a string to an integer, or a ZeroDivisionError. Print a friendly message.

EXAMPLE OUTPUT:

```
Could not convert 'abc' to an integer.
Please enter a valid number.
Exception caught: ValueError
```

```python
def convert_to_int(text):
    try:
        number = int(text)
        print(f"Converted number: {number}')
        result = 10 /  number
        print(f"10 / {number} = {result}')
    except ValueError:
        print(f"Could not convert '{text}' to an integer.')
        print("Please enter a valid number.')
    except ZeroDivisionError:
        print("Cannot divide by zero.')

# Example calls
convert_to_int("abc")    # Triggers ValueError
convert_to_int("0")      # Triggers ZeroDivisionError
```

SLIDE 33

finally

WHAT & WHY:

The finally block runs
no matter what.
Use it to clean up resources
like closing files or connections.

FINALLY ALWAYS RUNS.

It runs after try and except,
even if an error occurs.

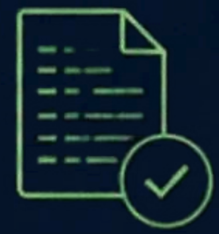
```
# Example: Read from a file and always close it
filename = "data.txt"

    f = open(filename, "r")
    content = f.read()
    print('"File contents:")
    print(content)
except FileNotFoundError:
   print('"Error: File not found.")
finally:
    f.close()
    print('"File closed.")

# The 'finally' block runs no matter what.
```

EXAMPLE OUTPUT:

```
File contents:
Hello, Python!
File closed.
```



TASK 08

PROMPT:

Ask for an integer and print it doubled. On ValueError, print one friendly message and retry exactly once. A valid second attempt should still be doubled.

EXAMPLE OUTPUT:

```
42 → Doubled: 84
hello → try again once

Hint: keep the conversion inside
try/except.
```

```python
# Ask for user input
user_input = input("Enter a number: ")

# Try to convert to integer
try:
    number = int(user_input)
    print(f"Doubled: {number * 2}")

except ValueError:
    print("That doesn't look like a whole number.")
    print("Please enter an integer.")
```

MODULES

DRY: Don’t Repeat Yourself

DRY is a principle that means writing code once and reusing it. It saves time, reduces errors, and makes your programs easier to maintain.

Reusable Code with Modules

Modules let you use code that others have already written—so you don’t have to reinvent the wheel.

Built-in Libraries

Come with Python and are ready to use.
No installation needed.

```
print(math.sqrt(16)                    # Output:  4.0
```

External Packages

Created by the Python community.
Install them to add extra features.

```
import  requests
response = requests.get("https://api.github.com")
print(response.status_code)          # Output:  200
```

Use modules to build better code, **faster**—so you can focus on **solving problems**, not rewriting solutions.

IMPORT SYNTAX

OVERVIEW:

Python provides multiple ways to import modules and specific objects.

IMPORT STYLES COMPARED:

import math

Imports the entire module under its namespace.

Use with module.

from datetime import datetime

Imports a specifie module directly into your namespace.

Use without module prefix.

```python
# Import the entire math module
import math

# Example usage
area = math.pi * (5 ** 2)
print(f"Area: {area}")
```

OUTPUT:

Area: 78.53981633974483

```python
# Import only the datetime class
from datetime import datetime

# Example usage
now = datetime.now()
print(f"Current time: {now}")
```

OUTPUT:

Current time: 2025-05-12 14:30:45.123456

TASK 09

PROMPT:

Generate 5 random integers from 1–100. Print the largest and the count of evens. No sorting; use one loop.

```python
# Import the random module
import random

# Generate and print a random number
print(random.randint(1, 100))
```

OUTPUT:

```
57
```

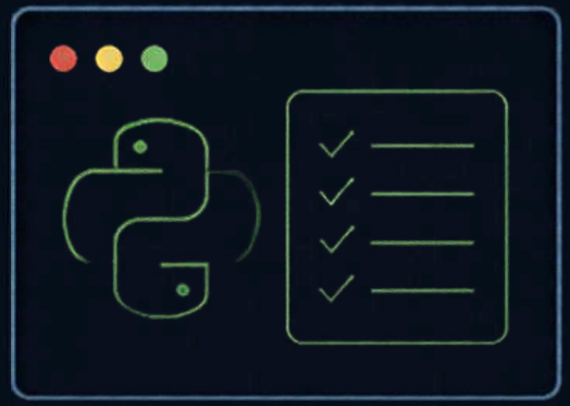
RECAP

1. PRIMITIVES
2. CONTROL FLOW
3. COLLECTIONS
4. FUNCTIONS
5. ERRORS
6. IMPORT



SLIDE 40

DATA STRUCTURES: RUNTIME CHEAT SHEET



The rehearsal module: quick Big-0 reference for common operations.

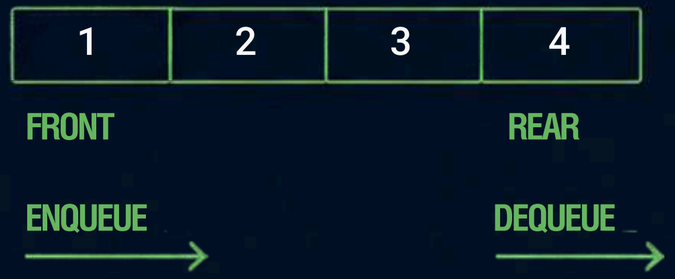
DATA STRUCTURE	ACCESS	SEARCH	INSERTION	DELETION	NOTES
Queue	O(1)	O(n)	O(1) (enqueue)	O(1) (dequeue)	FIFO order
Stack	O(1)	O(n)	O(1) (push)	O(1) (pop)	LIFO order
ArrayList	O(1)	O(n)	O(n) (avg: O(1)*	O(n) (avg: O(1)*	Contiguous memory, dynamic resizing
Linked List	O(n)	O(n)	O(1) (known node)	O(1) (known node)	Sequential access, no random access
Binary Tree	O(log n) (balanced)	O(log n) (balanced)	O(log n) (balanced)	O(log n) (balanced)	Worst case for skewed tree: O(n)
Hash Map	O(1) (avg)	O(1) (avg)	O(1) (avg)	O(1) (avg)	Worst case: O(n) (many collisions)

* Amortized (average) case. Worst-case can be O(n) due to resizing.

QUEUE VS STACK

QUEUE (FIFO)

First In, First Out



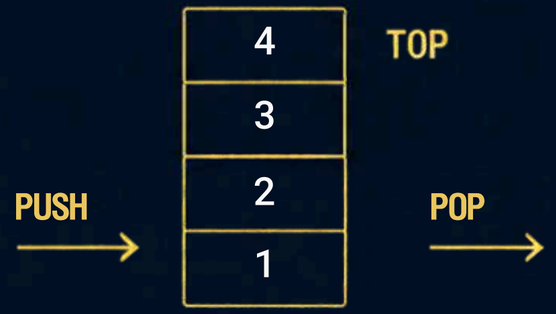
Typical Runtimes: O(1)

Common Uses:

- Print jobs
- Task scheduling
- Breadth-first search

STACK (LIFO)

Last In, First Out



Typical Runtimes: O(1)

Common Uses:

- Undo history
- Function calls
- Depth-first search

PYTHON EXAMPLE (CONCEPTUAL)

```
# Queue (FIFO) using collections.deque
from collections import deque
q = deque()
q = deque()     # enqueue
q.append(2)
front = q.popleft()    # deque -> returns 1
# q is now deque([2])

# Stack (LIFO) using list
s = []
s.append(1)     # push
s.append(2)
top = s.pop()  # pop -> returns 2
# s is now [1]
```

ARRAY / LIST VS LINKED LIST

PYTHON LIST (DYNAMIC ARRAY)

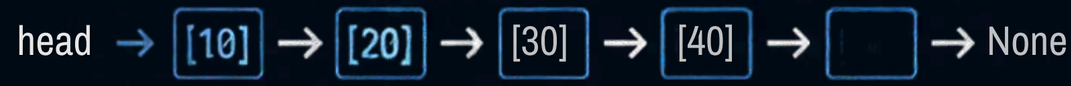
Indexed access	O(1)
Search	O(n)
Insertion / Deletion in the middle	O(n)

```
nums = [10, 20, 30, 40, 50]
x = nums[2]              # O(1)
nums.insert(2, 25)       # O(n)
```



LINKED LIST

Access by index	O(n)
Search	O(n)
Insertion / Deletion when node is known	O(1)




```
node = head.next          # points to 20
node.next = Node(25, node.next)  # insert 25 after 20 → O(1)
```

WHEN TO USE WHAT

- Contiguous storage
- Fast indexing (random access)
- Great general-purpose sequence

- Frequent inserts/deletes in the middle
- Splicing nodes when position is known
- No need for fast random access

BINARY TREES

PARENT-CHILD STRUCTURE

Each node has up to two children: a left child and a right child. One node (except the root) has exactly one parent.

COMMON USES

- Ordered data
- Search indexes
- Expression trees

BALANCED VS UNBALANCED BST



COMPLEXITY (BST OPERATIONS)

Operation	Balanced (Avg.) (1/3)	Unbalanced (Worst)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

CONCEPTUAL PYTHON EXAMPLE

```
class Node:
def _init_(self, val):
    self.val = val
    self.left = None
    self.right = None
```

Each node stores a value and links to left and right children.

HASH MAPS / DICTIONARIES

KEY-VALUE STORAGE

Store data as key-value pairs.
Keys are unique; values can be anything.

HASHING

A hash function converts the key to an index in a table (array) for fast access.

FAST LOOKUP

Hashes give (on average) constant-time lookup, insert, and delete.

LOOKUP (AVG)	INSERT (AVG)	DELETE (AVG)
O(1)	O(1)	O(1)

WORST CASE: O(n) due to hash collisions.

```python
# Python dictionary example
user = {
  "id": 101,
  "name": "Ada Lovelace",
  "role": "Engineer",
  "active": True
}

user["name"] # 'Ada Lovelace'
user["role"] = "Lead Engineer"
del user["active"]
```

COMMON USES

- Caches and memorization
- Counting frequencies
- Configuration and settings
- Working with JSON data
- Building APIs and request/response data

Keys must be hashtable (e.g., `str`, `int`, `tuple`).
Lists and `dicts` cannot be used as keys.

OBJECTS & CLASSES

A class is a blueprint.

It defines data (attributes) and behavior (methods).

An object is an instance.

It's a real item created from a class.

Classes connect data and behavior in one reusable type.

```python
class Player:

    def _init_(self, name):
    self.name = name

    def greet(self):
        return f"Hi, I'm {self.name}!"


# Create an object (instance)
player = Player("Ada")

# Access an attribute
print(player.name)

# Output: Ada
```



METHODS

Methods are functions defined inside a class.

self refers to the current object.

Methods let objects do things.

```python
class Player:
 def _init_(self, name, level=1):
 self.name  = name
 self.level = level

   def level_up(self):
    self.level += 1
    return f{self.name} reached level {self.level}!"

# Create an object
player = Player("Ada", level=1)

# Call a method
message = player.level_up()

# Output
print(message)

# Output: Ada reached level 2!
```



CONSTRUCTORS

The `__init__` method is the constructor.

It runs automatically when an object is created.

Constructors set the initial state of an object by initializing its attributes.

```python
class Player:

    def _init_(self, name, score):
        self.name = name
        self.score = score

# Create an object (constructor runs here)
player = Player("Ada", 95)

# Access an attribute
print(player.name)

# Output:  Ada
```



POLYMORPHISM

Different classes can expose the **same method** name with different **behavior**.

The caller does **not** need to know the exact class-only that the object **supports the method**.

```python
class Dog:
    def speak(self):
        return "Woof!"

class Cat:
    def speak(self):
        return "Meow!"

# Polymorphic usage
animals = [Dog(), Cat()

# Caller works with any object
that # has a speak() method
for animal in animals:
    print(animal.speak()
```



```
# Output:
Woof!
Meow!
```

EXTERNAL APIS

An API lets a Python program communicate with another service over HTTP.

1 → 3 → 5

Build Request
Set endpoint, params, headers, and (optional) body.

Send
Make the HTTP request to the API server.

Receive Response
The server responds with status and data.

Parse JSON
Convert the JSON response into Python data.

Use Data
Work with the data in your program.

GET

GET Requests
Use GET to retrieve data. Pass parameters in the URL query string.

200

Status Codes
Check the status code. 200 = OK, 4xx = client error, 5xx = server error.

{ }

JSON
Most APIs return JSON. Parse it to access fields and values.

API Keys
Keep API keys safe. Store them in environment variables, not in code.

```
import os
API_KEY = os.getenv("API_KEY")
```

CALLING AN API IN PYTHON

```python
import requests

response = requests.get(
) "https://api.example.com/weather?city=London"

response.raise_for_status()

data = response.json()

print(data["temperature"])
```

Note: The URL is illustrative. Real APIs may require authentication
JSO1 keys, tokens, headers) and have rate limits.

`requests.get(url)`
Sends an HTTP GET request to the specified URL and returns a response object.

`raise_for_status()`
Checks the response status code and raises an error for bad responses (4xx or 5xx).

`response.json()`
Parses the response body as JSON and returns a Python dictionary (data).



FINAL TASK

Build a Mini Library System

Create a small object-oriented program that models books and a library.



REQUIREMENTS

1. Create a **Book** class with title, author, and **checked_out** state.
2. Create a **Library** class that stores books.
3. Add methods to:
   - add a book
   - search by title
   - check out a book
   - return a book
4. Handle attempts to check out a missing or already checked-out book with **friendly messages**.
5. Print a short **borrowing report**.

CHALLENGE CONSTRAINT

Use at least one shared method name or **polymorphic** behavior across classes.

Include a small API-inspired feature:

Export the report as a **dictionary** ready for JSON (no file I/O required).

SUGGESTED TEST SEQUENCE

1. Create a **Library**.
2. Add a few books.
3. **Search** for an existing title.
4. **Search** for a missing title.
5. Check **out** an existing book.
6. Try to check out the same book again.
7. Try to check out a missing book.
8. **Return** the book.
9. Print the borrowing report and show the exported **dictionary**.

HINT

Think about responsibilities: **Book** knows its own state; **Library** manages a collection of books.
Search case-insensitively. Keep messages clear and helpful.
For API-style export, return a **dictionary** with library name, total books, **checked out** count, and a list of books with their status.

```
report = library.exportReport()

# Ready for JSON:
# json.dumps report)
```

NEXT STEPS

Upload your class project to GitHub using the commands from Session 1! See you next week.

